

Михаил Трифонов

Бэкенд-разработчик, Java и Python

T-Банк, Data Platform



trifonov2812@gmail.com

t.me/LuminiteTime

github.com/LuminiteTime

luminais.tech

Пишу бэкенд на Java и Spring Boot. Сейчас в Т-Банке делаю сервисы платформы данных на Kora и Python. До этого в Яндексе писал BFF для отзывов Auto.ru, а в Hirus делал бэкенд «Умной операционной».

В свободное время довожу собственные продукты до продакшена. Один из них, Skillator, поработал три месяца с платящими пользователями, и я его продал. Здесь собраны три проекта, где архитектура целиком на мне, и хакатоны, на которых мы брали призы.

Где работал

Т-Банк	Data Platform, бэкенд на Java и Python	с сентября 2025
Яндекс, Auto.ru	стажёр, бэкенд на Scala	с января по апрель 2025
Hirus	бэкенд «Умной операционной» на Java	с июля 2023 по август 2024
Иннополис	бакалавриат Computer Science, GPA 4.77	выпуск в 2027

В этом файле

Pojec	платформа для найма, где кандидат чинит настоящий сервис	стр. 2
Skillator	платный AI-сервис, который я сделал один и продал	стр. 3
Вместе	закрытое приложение для семьи с ассистентом	стр. 4
Хакатоны	MTS True Tech Hack, AdventureX, T1 и другие	стр. 5

Ројес

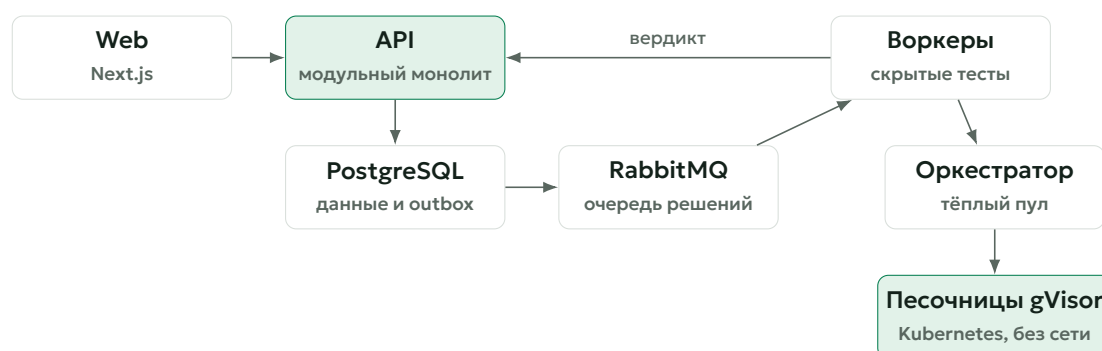
Платформа для найма, где кандидат чинит настоящий сервис.

Основатель и техлид. С января 2026. Я и два контрибьютора.

Задача

На собеседованиях до сих пор решают алгоритмические задачки, хотя в работе важнее другое. Нужно разобраться в чужом коде, подключить фреймворк и ничего не сломать в проде. В Ројес кандидат получает сервис с багом или недоделкой, чинит его, а скрытые тесты проверяют решение в песочнице.

Сложность в том, что платформа запускает чужой код. Во время найма на неё приходят всплески, и каждое решение нужно проверить ровно один раз.



Решения

Модульный монолит. Начал с одного приложения на Spring Boot и одной PostgreSQL. Для команды из трёх человек распределённые транзакции были бы лишней болью. Границы модулей проверяют тесты на ArchUnit, так что любой модуль потом можно вынести в отдельный сервис.

Песочницы на gVisor. Код кандидата запускается в gVisor внутри Kubernetes. Выхода в сеть у песочницы нет, а ресурсы и время запуска жёстко ограничены.

Outbox и RabbitMQ. Решение и событие о нём пишутся в базу одной транзакцией, отдельный процесс перекладывает события в очередь. Падение воркера не теряет запуск и не проверяет его дважды. По глубине очереди Kubernetes добавляет песочницы.

Тёплый пул. Половину времени до вердикта съедал холодный старт песочницы. Теперь заранее держится пул готовых песочниц, его размер считается от потока решений.

Как работаем

Работаем двухнедельными циклами в Linear. Раз в неделю провожу 1:1 с каждым контрибьютором. Изменение, которое затрагивает несколько модулей, сначала оформляется в ADR и проходит дизайн-ревью.

14 с

р95 до вердикта при 300
одновременных решениях в тесте
k6

0

потерянных запусков из 50
000, когда воркеры убивались
случайно

22

архитектурных решения записаны
в ADR

Java 25, Spring Boot, PostgreSQL, RabbitMQ, Kubernetes, gVisor, Next.js, k6

Skillator

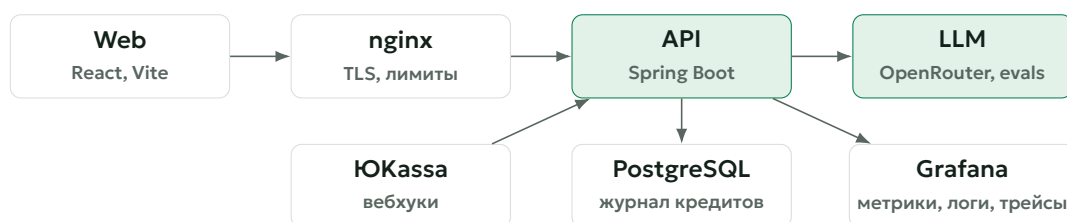
Платный AI-сервис, который я сделал один и продал.

Основатель и единственный инженер. С июня по сентябрь 2026.

Задача

Skillator по описанию задачи пишет скилл для кодовых агентов вроде Claude Code. Генерация платная, пользователь покупает кредиты через ЮKassa. Я сделал его сам, от интервью с первыми пользователями до деплоя, три месяца держал в продакшене и потом продал.

Главная трудность в том, что в одном потоке оказались реальные деньги и недетерминированная модель. Списание должно быть верным всегда, а ответ должен стоить своих денег.



Решения

Кредиты как журнал. Баланс нигде не хранится числом, которое можно поправить. Каждое пополнение и каждая генерация пишутся отдельной неизменяемой записью, а баланс считается из них. Вебхуки ЮKassa идемпотентны по id платежа, так что повторное уведомление не начислит кредиты второй раз.

Evals перед релизом. Любое изменение промпта или модели прогоняется по эталонному набору запросов. Ответы оценивает LLM-судья по рубрике, и если оценка упала, релиз не выходит.

Провайдеры за интерфейсами. Модель и платёжка переключаются со страницы администратора. Когда у провайдера модели был сбой, я переключил его за минуту без деплоя.

Мониторинг с первого дня. Метрики шли в Prometheus, логи в Loki, трейсы в Tempo. Алерты о проблемах с оплатой приходили мне в Telegram.

Передача покупателю. К продаже подготовил ранбуки, ADR и всю инфраструктуру в коде. Через неделю команда покупателя сама выпускала релизы.

2 300

пользователей, из них 310
платящих

99,9%

аптайм за три месяца в
продакшене

16

архитектурных решений записаны
в ADR

Java 25, Spring Boot, PostgreSQL, React, ЮKassa, Docker, Prometheus, Grafana, Loki, Tempo

Вместе

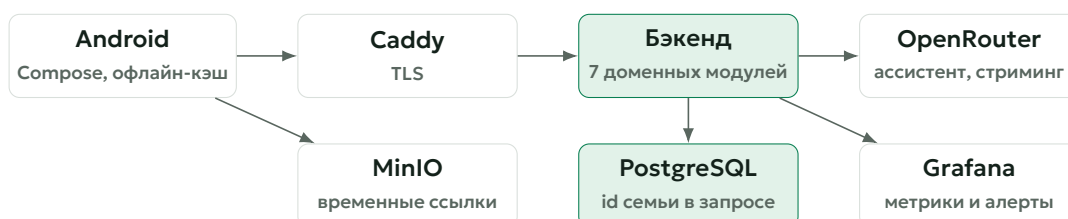
Закрытое приложение для семьи с ассистентом.

Автор, бэкенд и Android. С августа 2026.

Задача

В приложении есть общий чат, документы, календарь и ассистент. Ассистент отвечает на вопросы по данным самой семьи, например где лежит полис или когда у бабушки день рождения.

На одном сервере живут несколько семей. Главное требование в том, чтобы они никогда не видели данные друг друга, и ассистент тоже.



Решения

Изоляция в репозиториях. Id семьи берётся только из токена и подставляется в каждый запрос на уровне репозитория. Выбрать чужую семью через тело запроса нельзя в принципе.

Файлы мимо бэкенда. Приложение получает временную ссылку и грузит файл прямо в MinIO. Бэкенд большие файлы через себя не гоняет.

Смена поиска. Сначала ассистент искал по pgvector. Потом оказалось, что у провайдера модели нет эмбеддингов, и я перешёл на лексический поиск. Почему и при каком условии вернуться, записано в ADR.

Офлайн на Android. Клиент на Kotlin и Compose читает из локального кэша. Отправленные сообщения ждут сети в очереди и уходят сами.

Релизы. Релиз запускается вручную в GitHub Actions. Галочками выбирается, что выкатывать, APK, сервер или мониторинг.

7

доменных модулей, границы
проверяет компилятор

11

архитектурных решений записаны
в ADR

300 мс

бюджет р95 на REST-запрос

Java 25, Spring Boot 4, Spring Data JDBC, PostgreSQL, MinIO, Kotlin, Jetpack Compose, Caddy

Хакатоны и учебные проекты

Короткие проекты, где я отвечал за бэкенд.

2025 и 2026 годы

MTS True Tech Hack, лучшее решение в треке MWS Tables

Апрель 2026. React, TypeScript, Java, Spring, Node.js, PostgreSQL

Сделали редактор документов в духе Confluence, в который встраиваются живые таблицы MWS Tables. Я писал бэкенд метаданных на Java и интеграцию с таблицами. Чтобы сервис совместного редактирования на Node.js не зависел от Java-бэкенда, связал их через очередь событий в PostgreSQL.

AdventureX, Ханчжоу

Июль 2026. React Native, Expo, TypeScript, Go, vLLM

Самый большой хакатон Китая, больше 800 участников. За пять дней вчетвером сделали The Sleepy Knight, пиксельного тамагочи, который живёт, только если ты спишь по режиму. Я написал Go-сервис для AI-оракула игры и сделал часть экранов на React Native.

T1 Hackathon, PingTower

Сентябрь 2025. Java 21, Spring Boot, ClickHouse, Python

Мониторинг доступности сайтов и API с алертами и отчётами по SLA. Я был лидером бэкенда и разделил систему на два сервиса. Проверки и аналитика живут на Java с ClickHouse, уведомления отправляет сервис на Python.

Techarena Kazan, 2 место

Июнь 2025. C++

Алгоритм распределения сетевого потока между узлами с ограничениями по пропускной способности и цене. Я отвечал за проверку гипотез и краевые случаи.

Open Labs Share

Лето 2025. Spring Cloud, gRPC, FastAPI, PostgreSQL

Учебный проект на семь человек, я вёл бэкенд. Делил задачи между тремя бэкенд-разработчиками, проводил дизайн-ревью и сам написал авторизацию и gateway.

Компилятор языка I

Весна 2025. Java, JavaCC, WebAssembly

Компилятор своего императивного языка в WebAssembly. Парсер сделан на JavaCC, семантический анализ и генерация кода написаны вручную, программы запускаются через wasmtime.